PYTHON CHEAT SHEET · 04 OF 05

Data Types in Python

Built-in types, mutability, conversion and truthiness, plus the gotchas that trip people up.

TYPING	EVERYTHING IS	MUTABLE	IMMUTABLE
Dynamic and strong	An object with a type	list, dict, set, bytearray	int, float, str, tuple, bytes

01 The built-in type map

Category	Type	Mutable?	Example
Numeric	int	No	`42 1_000_000 0xFF`
Numeric	float	No	`3.14 1e-3`
Numeric	complex	No	`2 + 3j`
Boolean	bool (subclass of int)	No	`True False`
Text	str	No	`"hello" 'hi'`
Sequence	list	Yes	`[1, 2, 3]`
Sequence	tuple	No	`(1, 2, 3)`
Sequence	range	No	`range(0, 10, 2)`
Mapping	dict	Yes	`{"a": 1}`
Set	set	Yes	`{1, 2, 3}`
Set	frozenset	No	`frozenset({1, 2})`
Binary	bytes	No	`b"hi"`
Binary	bytearray	Yes	`bytearray(b"hi")`
None	NoneType	No	`None`

02 Checking and converting types

```
>>> [type(x).__name__ for x in (42, 3.14, 1 + 2j, "hi", [1], (1,), {1}, {"a": 1}, None)]
['int', 'float', 'complex', 'str', 'list', 'tuple', 'set', 'dict', 'NoneType']
>>> isinstance(5, int)
True
>>> isinstance(True, int)                          # bool IS an int
True
>>> isinstance(5, (int, float))                    # tuple of types: any match
True
```

```
>>> int("42") + 1
43
>>> float("1e3")
1000.0
>>> str(42) + "!"
'42!'
>>> list("abc"), tuple([1, 2]), set([1, 1, 2])
(['a', 'b', 'c'], (1, 2), {1, 2})
>>> dict([("a", 1)])
{'a': 1}
>>> ord("A"), chr(97)
(65, 'a')
>>> bin(10), hex(255), oct(8)
('0b1010', '0xff', '0o10')
```

```
>>> int("3.7")
ValueError: invalid literal for int() with base 10: '3.7'
>>> int(float("3.7"))                                  # int() truncates toward zero
3
>>> "5" + 5
TypeError: can only concatenate str (not "int") to str
>>> "5" * 3                                            # strong typing: no silent conversion
'555'
```

03 Numbers: int, float, complex, bool

```
>>> 7 / 2, 7 // 2, 7 % 2, 2 ** 10                      # true div, floor div, remainder, power
(3.5, 3, 1, 1024)
>>> divmod(17, 5)
(3, 2)
>>> -7 // 2                                            # floor division rounds toward -infinity
-4
>>> 2 ** 100                                           # int has no size limit
1267650600228229401496703205376
>>> 1_000_000, 0b1010, 0o17, 0xFF                      # underscores and base prefixes
(1000000, 10, 15, 255)
>>> (2 + 3j) * (1 - 1j)
(5+1j)
>>> True + True                                        # bool is a number: True == 1
2
```

```
>>> 0.1 + 0.2
0.30000000000000004
>>> 0.1 + 0.2 == 0.3                                   # binary floating point is approximate
False
>>> import math
>>> math.isclose(0.1 + 0.2, 0.3)                       # compare floats with a tolerance
True
>>> from decimal import Decimal
>>> Decimal("0.1") + Decimal("0.2")                    # exact decimal arithmetic (money!)
Decimal('0.3')
>>> round(2.5), round(3.5), round(3.14159, 2)          # ties round to the nearest EVEN number
(2, 4, 3.14)
```

04 Strings (str): immutable text

```
>>> s = "Python Rocks"
>>> s[0], s[-1], s[0:6], s[::-1]                    # index, last, slice, reversed
('P', 's', 'Python', 'skcoR nohtyP')
>>> s.upper(), s.lower(), s.title()
('PYTHON ROCKS', 'python rocks', 'Python Rocks')
>>> s.replace("Rocks", "Rules")                     # returns a NEW string
'Python Rules'
>>> s.split(" ")
['Python', 'Rocks']
>>> ", ".join(["a", "b", "c"])
'a, b, c'
>>> "  padded  ".strip()
'padded'
>>> s.find("R"), s.startswith("Py"), "thon" in s
(7, True, True)
>>> name, score = "Asha", 91.456
>>> f"{name} scored {score:.1f}"                    # f-string with format spec
'Asha scored 91.5'
>>> f"{1234567:,}"                                  # thousands separator
'1,234,567'
>>> f"{7:03d}|{'hi':>6}|{0.256:.1%}"                # zero-pad, right-align, percent
'007|    hi|25.6%'
>>> r"C:\new\table"                                # raw string: backslashes stay literal
'C:\\new\\table'
>>> s[0] = "J"
TypeError: 'str' object does not support item assignment
```

05 list, tuple and range

```
>>> nums = [5, 2, 8]
>>> nums.append(1)
>>> nums.extend([9, 7])
>>> nums.insert(0, 100)
>>> nums.remove(2)                                  # removes the first VALUE equal to 2
>>> nums.pop()                                      # removes and returns the last item
7
>>> nums
[100, 5, 8, 1, 9]
>>> sorted(nums), nums[::2], nums[-2:]              # sorted() returns a new list
([1, 5, 8, 9, 100], [100, 8, 9], [1, 9])
>>> nums.sort(reverse=True)                         # sort() changes the list in place
>>> nums
[100, 9, 8, 5, 1]
>>> t = (1, 2, 3)
>>> type((42)), type((42,))                         # the comma makes the tuple
(<class 'int'>, <class 'tuple'>)
>>> first, *rest = t                                # starred unpacking
>>> first, rest
(1, [2, 3])
>>> t[0] = 9
TypeError: 'tuple' object does not support item assignment
>>> list(range(2, 11, 3))                           # start, stop (excluded), step
[2, 5, 8]
>>> [n * n for n in range(5) if n % 2 == 0]         # list comprehension
[0, 4, 16]
```

06 Truthiness and None

Falsy values (everything else is truthy)	Examples
Constants	`None`, `False`
Zero of any numeric type	`0`, `0.0`, `0j`
Empty containers and text	`""`, `[]`, `()`, `{}`, `set()`, `range(0)`

```
>>> [bool(x) for x in (0, 0.0, "", [], {}, set(), None, "0", [0], " ")]
[False, False, False, False, False, False, False, True, True, True]
>>> bool("False")                                    # any non-empty string is truthy
True
>>> x = None
>>> x is None                                        # test None with `is`, not `==`
True
```

07 Mutability, identity and copies

```
>>> a = [1, 2]
>>> b = a                                            # two names, ONE list
>>> b.append(3)
>>> a
[1, 2, 3]
>>> c = a.copy()                                     # an independent (shallow) copy
>>> c.append(4)
>>> a, c
([1, 2, 3], [1, 2, 3, 4])
>>> a is b, a is c, a == c                           # `is` = same object, `==` = same value
(True, False, False)
```

```
>>> def add_item(item, bucket=[]):                   # BAD: default list is created once
...     bucket.append(item)
...     return bucket
>>> add_item(1), add_item(2)
([1, 2], [1, 2])
>>> def add_item_fixed(item, bucket=None):           # GOOD: build a fresh list per call
...     if bucket is None:
...         bucket = []
...     bucket.append(item)
...     return bucket
>>> add_item_fixed(1), add_item_fixed(2)
([1], [2])
```

WATCH OUT Never use a mutable object (`[]`, `{}`, `set()`) as a default argument value. Use `None` and create the object inside the function.

08 Type hints (not enforced at runtime)

```
>>> def double(n: int) -> int:
...     return n * 2
>>> double(4)
8
>>> double("ab")                                     # hints are NOT enforced at runtime
'abab'
>>> names: list[str] = ["a", "b"]
>>> scores: dict[str, int | None] = {"a": 1, "b": None}
```

Use a checker such as `mypy` or `pyright` to catch type-hint violations before running the code.